An Improved Lower Bound for the Chvátal–Sankoff Constant via Windowed Dynamic Programming

Yukachev Dmitriy Evgenevich
HSE University — St. Petersburg, School of Informatics, Physics and Technology
Educational Programme "Applied Mathematics and Computer Science"
Supervisor: Kopeliovich S. V.

2025

Abstract

This paper proposes a computational method for obtaining rigorous lower bounds on the Chvátal–Sankoff constant γ_2, which governs the asymptotic behaviour of the expected length of the longest common subsequence (LCS) of two random binary strings of length n. The method is based on dynamic programming with a bounded lookahead window: at each step the algorithm decides how to extend a common subsequence using information about the next N symbols of both strings. The bounded horizon guarantees that the strategy found cannot outperform the optimal one, thereby yielding a valid lower bound. Large-scale numerical experiments have been carried out, producing the estimate $\gamma_2 \geq 0.7964$, which exceeds the best published result of 0.792666 (Heineman et al., 2024). An independent verification via a Monte Carlo method has also been performed, giving the empirical estimate $\gamma_2 \approx 0.811$. **Keywords:** Chvátal–Sankoff constant, longest common subsequence, LCS, dynamic programming, lower bound, random binary strings, Monte Carlo method.

1 Introduction

The Longest Common Subsequence (LCS) problem is one of the fundamental problems in combinatorics and theoretical computer science. Given two strings X and Y, their LCS is the longest sequence of symbols that is a subsequence of both strings simultaneously. A subsequence is obtained from the original string by deleting some (possibly zero) elements without changing the order of the remaining ones. Computing the LCS length by the classical dynamic programming algorithm takes $O(n^2)$ time [10] and underlies numerous applications, ranging from biological sequence alignment in bioinformatics to file comparison algorithms (the `diff` utility) and version control systems [1].

Beyond its practical applications, the LCS problem for random strings is of considerable theoretical interest. It is closely related to the problem of increasing subsequences of random permutations (Ulam's problem) and to random matrix theory via the Tracy–Widom distribution [8].

In 1975, Chvátal and Sankoff [1] posed the question about the behaviour of the LCS length for two *random* strings. Let X and Y be two independent random strings of length n over the alphabet $\Sigma = \{0, 1\}$, where each symbol is chosen uniformly and independently at random. Let L_n denote the length of their LCS. Chvátal and Sankoff showed that the limit

$$\gamma_2 \;=\; \lim_{n\to\infty} \frac{\mathbb{E}[L_n]}{n}$$

exists; it is called the *Chvátal–Sankoff constant* for the binary alphabet. Despite half a century of research, the exact value of γ_2 remains unknown. The best known bounds to date are:

- lower bound: $\gamma_2 \geq 0.792666$ (Heineman et al., 2024 [6]);
- upper bound: $\gamma_2 \leq 0.826280$ (Lueker, 2009 [5]).

The gap between the known bounds is approximately 0.034, making the task of narrowing this interval relevant from both theoretical and computational perspectives. The difficulty of the problem is related, in particular, to the fact that no closed-form expression for $\mathbb{E}[L_n]$ exists even for small n, and the available combinatorial methods quickly run into the exponential growth of the number of configurations.

Objective. The aim of this work is to obtain an improved rigorous lower bound for the constant γ_2 that surpasses the best known results. To achieve this objective, the following **tasks** have been set:

1. Develop a computational method for estimating $\mathbb{E}[\mathrm{LCS}]$ based on dynamic programming with a bounded lookahead window.
2. Implement the method in C++ with optimisations that enable handling large parameter values.
3. Conduct large-scale numerical experiments to obtain lower bounds for various parameter settings.
4. Perform an independent verification of the results using a Monte Carlo method.
5. Compare the obtained results with the best known bounds from the literature.

2 Literature Review

2.1 Origin of the Problem and Early Results

The problem was posed by Chvátal and Sankoff in 1975 [1]. They proved the existence of the limit $\gamma_k = \lim_{n\to\infty} \mathbb{E}[L_n]/n$ for an alphabet of size k and established the first lower bounds: $\gamma_2 \geq 0.7615$ for the binary alphabet. This result was obtained by means of a constructive argument: the authors exhibited a specific (suboptimal) strategy for building a common subsequence and computed its average payoff.

The existence of the limit itself follows from Kingman's subadditive lemma: the quantity $\mathbb{E}[L_{m+n}] \geq \mathbb{E}[L_m] + \mathbb{E}[L_n]$ is superadditive, which guarantees the existence of the limit $\mathbb{E}[L_n]/n$.

Dančík [2] developed the theory of lower bounds by proposing a systematic approach based on finite automata. His method relies on the idea of Markov strategies: a strategy

decides whether to include a symbol in the LCS based on a finite state that depends on the local context of the strings. The quality of the bound depends on the number of states of the automaton—the more states, the better the strategy approximates the optimal one, but the higher the computational cost. In a joint work with Paterson [3], the bound $\gamma_2 \geq 0.773911$ was obtained; it remained the best for a long time.

2.2 Methods for Upper Bounds

Upper bounds require fundamentally different techniques. Lueker [4] in 2003 obtained the lower bound $\gamma_2 \geq 0.788071$ using optimised Markov strategies with a large number of states. Later, in 2009, the same author [5] established the upper bound $\gamma_2 \leq 0.826280$, which is currently the best known upper bound. Lueker's method for upper bounds is based on analysing the superadditivity of the sequence $\mathbb{E}[L_n]$ and combinatorial inequalities.

2.3 Recent Results

Heineman et al. [6] in 2024 improved the lower bound to $\gamma_2 \geq 0.792666$. Their approach employs a large-scale computational search over strategies for constructing common subsequences, optimised using techniques from the theory of finite automata, and required substantial computational resources. This result constitutes the best published lower bound at the time our study began.

Tiskin [7] in 2023 proposed viewing the Chvátal–Sankoff problem as a problem of symbolic dynamics, opening new theoretical approaches to analysing the constant γ_2 through the study of properties of the tropical matrix semigroup.

For larger alphabet sizes, Kiwi, Loebl, and Matoušek [9] established asymptotic estimates of γ_k as $k \to \infty$, showing that $\gamma_k = 1 - O(1/\sqrt{k})$. This indicates that the binary case ($k = 2$) is the most difficult to analyse.

2.4 Exact Values for Short Strings

For short strings of length n, one can compute $\mathbb{E}[L_n]/n$ by exhaustive enumeration of all 4^n pairs of strings. For each pair, the LCS length is computed by the standard dynamic programming algorithm in $O(n^2)$ time, and the result is then averaged. The overall complexity is $O(4^n \cdot n^2)$, which limits computations to values $n \leq 14$–20. In this work, exact values for $n \leq 14$ have been obtained (see Table 4) and are used to verify the main method. The sequence $\mathbb{E}[L_n]/n$ is monotonically increasing and, according to theoretical results, converges to γ_2.

3 Methods

3.1 Main Idea: A Greedy Strategy with Bounded Lookahead

The key observation underlying the method is as follows. Any fixed *strategy* for constructing a common subsequence of two random strings yields a lower bound on $\mathbb{E}[\mathrm{LCS}]$. Indeed, the strategy produces some common subsequence (not necessarily the longest one), and the length of the subsequence found does not exceed the LCS length. Consequently, the average length of the subsequence produced by the strategy does not exceed $\mathbb{E}[L_n]$.

Proposition 1. *Let $\mathcal{S}$ be an arbitrary strategy for constructing a common subsequence of two strings, and let $C_n^{\mathcal{S}}$ denote the length of the subsequence produced by strategy $\mathcal{S}$ for a pair of random strings of length n. Then $\mathbb{E}[C_n^{\mathcal{S}}] \leq \mathbb{E}[L_n]$, and in particular,*

$$\liminf_{n\to\infty} \frac{\mathbb{E}[C_n^{\mathcal{S}}]}{n} \;\leq\; \gamma_2\,.$$

This proposition follows directly from the definition of LCS as the *longest* common subsequence: for each specific pair of strings $C_n^{\mathcal{S}} \leq L_n$, whence the inequality for the expectations follows.

Our method defines a strategy with a bounded lookahead horizon and computes the average length of the subsequence built by this strategy, averaging over all possible continuations of the strings.

3.2 Dynamic Programming with Windows

Consider two binary strings $A = a_1a_2\ldots$ and $B = b_1b_2\ldots$, generated symbol by symbol. At each step, the strategy "sees" the next N symbols of both strings and makes one of the following decisions: advance the position in string A, advance the position in string B, or, if the current symbols match, take the symbol into the common subsequence.

We formalise this as a dynamic programming problem. A state is described by a quadruple $(L, \mathit{shift}, \mathit{maskA}, \mathit{maskB})$, where:

- L is the remaining number of steps (the length of the unprocessed portion of the strings);
- $\mathit{shift} \in \{0, 1, \ldots, MAX\_SH\}$ is the current difference in positions $|i - j|$ between the pointers in strings A and B;
- $\mathit{maskA} \in \{0,1\}^N$ is the bitmask of the next N symbols of string A;
- $\mathit{maskB} \in \{0,1\}^N$ is the bitmask of the next N symbols of string B.

The value $\mathrm{dp}[L][\mathit{shift}][\mathit{maskA}][\mathit{maskB}]$ represents the expected length of the common subsequence that can be built from the remaining L symbols under the given configuration.

Transitions are defined as follows. If the first visible symbols match ($\mathit{maskA}[0] = \mathit{maskB}[0]$), the strategy takes the symbol into the subsequence, receiving a payoff of 1 and advancing both positions. If the symbols do not match, the strategy chooses the better of two options: advance the position in A (decreasing *shift*) or advance the position in B (increasing *shift*). When advancing a position, the next symbol of the string is not yet known; therefore, averaging is performed over both possible values $\{0, 1\}$ with equal probabilities.

Transition table:

Action	**Condition**	ΔL	$\Delta \mathit{shift}$
Take symbol into CS	$\mathit{maskA}[0] = \mathit{maskB}[0]$	-1	0
Advance A	$\mathit{shift} > 0$	-1	-1
Advance B	$\mathit{shift} < MAX\_SH$	0	$+1$
Swap A and B	$\mathit{shift} = 0$	0	$+1$

The iteration is performed over L from 1 to K, where K is the target string length. The final estimate is the average value of $\mathrm{dp}[K][0][\mathit{maskA}][\mathit{maskB}]$ over all pairs of masks, normalised by K.

3.3 Algorithm Complexity

The state space consists of $MAX\_SH \times 2^N \times 2^N = MAX\_SH \cdot 4^N$ states. For each state, averaging is performed over 4 possible continuations (2 variants of the new bit in $A \times 2$ variants of the new bit in B).

- **Memory:** $O(MAX\_SH \cdot 4^N)$ `float` values.
- **Time:** $O(K \cdot MAX\_SH \cdot 4^N)$ arithmetic operations.

For the parameters $N = 12$, $MAX\_SH = 30$, $K = 40{,}000$, the total number of operations is on the order of $2 \cdot 10^{13}$.

3.4 Implementation Optimisations

The algorithm went through several optimisation iterations (versions x0–x5):

1. **Separation of parameters N and MAX_SH** (x3): in earlier versions, the window size and the maximum shift were coupled, which restricted the parameter space.
2. **Iterative approach** (x3–x5): replacing recursion with iteration over L eliminated function call overhead and allowed the use of only $O(1)$ layers in L.
3. **Simplified mask representation** (x5): switching to a representation in which both masks are stored in N-bit words (without shifts), which doubled the speed and halved the memory consumption.
4. **Compiler directives:** use of `-Ofast`, AVX2 instructions, and loop unrolling.
5. **Use of `float` instead of `double`:** testing confirmed that 5–6 significant digits of precision are sufficient for this problem; meanwhile, memory consumption is halved and speed increases due to SIMD processing.

3.5 Verification via Monte Carlo Method

For independent verification of the results, a Monte Carlo method was implemented: pairs of random binary strings of a given length are generated, the LCS is computed for each pair using the classical $O(n^2)$ algorithm, and the ratio L_n/n is then averaged. Experiments were conducted for a wide range of string lengths, from $n = 50$ to $n = 20{,}000$, with the number of trials decreasing as n grows due to the quadratic complexity of computing each LCS. The standard deviation of the estimate $\mathbb{E}[L_n]/n$ decreases as $O(1/\sqrt{T})$, where T is the number of trials, allowing the precision of the results to be controlled.

It is important to emphasise that the Monte Carlo method and the windowed DP method provide fundamentally different types of estimates. The Monte Carlo method computes the *exact* LCS for each pair of strings (rather than the result of a suboptimal strategy); therefore, the resulting empirical estimate is an unbiased estimator of $\mathbb{E}[L_n]/n$ for a given n. In contrast, the windowed DP method computes the length of the common subsequence found by a strategy with bounded lookahead, yielding a rigorous lower bound. The results show convergence of $\mathbb{E}[L_n]/n$ to a value of ≈ 0.811 as $n \to \infty$ (see Section 4), which is experimentally confirmed by the inequality $0.7964 < 0.811$.

4 Results

4.1 Lower Bounds via the Windowed DP Method

Table 1 presents the experimental results for various parameter settings. The best result

Table 1: Lower bounds on γ_2 for various windowed DP parameters

Version	N	MAX_SH	K	Lower bound on γ_2
x2	7	7	1,000	≥ 0.7779
x3	7	30	5,000	≥ 0.7843
x4	7	50	30,000	≥ 0.7870
x4	8	50	50,000	≥ 0.7899
x4	9	60	100,000	≥ 0.7902
x5	10	35	100,000	≥ 0.7936
x5	11	35	20,000	≥ 0.7944
x5	12	30	40,000	$\geq \mathbf{0.7964}$

obtained is $\gamma_2 \geq 0.7964$ with parameters $N = 12$, $MAX\_SH = 30$, $K = 40{,}000$. As can be seen from the table, increasing the window size N has the greatest impact on the quality of the bound: each unit increase in N leads to a noticeable improvement in the lower bound, since the strategy receives more information for decision-making. Increasing the parameter K (string length) also improves the bound, as boundary effects become less significant for larger K.

4.2 Comparison with the Best Known Bounds

Table 2: Timeline of lower bounds on γ_2

Authors	**Year**	**Lower bound**
Chvátal, Sankoff	1975	≥ 0.7615
Dančík, Paterson	1995	≥ 0.7739
Lueker	2003	≥ 0.7881
Heineman et al.	2024	≥ 0.7927
This work	**2025**	$\geq \mathbf{0.7964}$

The obtained bound exceeds the best published result by ≈ 0.004. It should be noted that progress on lower bounds over the past decades has been slow: an improvement of 0.004 is comparable to the cumulative progress from 2003 to 2024 (from 0.7881 to 0.7927).

4.3 Empirical Estimate via the Monte Carlo Method

The Monte Carlo data confirm that $\gamma_2 \approx 0.811$, while the lower bound of 0.7964 leaves room for further improvement. The gap between the DP-method lower bound and the Monte Carlo empirical estimate is ≈ 0.015, indicating the possibility of substantial further improvement of the lower bound by increasing the algorithm's parameters.

Table 3: Empirical values of $\mathbb{E}[L_n]/n$ (Monte Carlo method)

n	$\mathbb{E}[L_n]$	$\mathbb{E}[L_n]/n$	Number of trials
50	38.3	0.766	10,000
100	78.1	0.781	5,000
500	400.4	0.801	2,000
1,000	804.8	0.805	1,000
5,000	4,048	0.810	300
10,000	8,104	0.810	100
20,000	16,220	0.811	50

4.4 Exact Values for Short Strings

Table 4: Exact values of $\mathbb{E}[L_n]/n$ (exhaustive enumeration)

n	$\mathbb{E}[L_n]/n$	n	$\mathbb{E}[L_n]/n$
2	0.5625	9	0.6913
3	0.6042	10	0.6978
4	0.6309	11	0.7035
5	0.6492	12	0.7085
6	0.6633	13	0.7129
7	0.6745	14	0.7168
8	0.6836		

5 Conclusion

In this work, a dynamic programming method with a bounded lookahead window has been proposed and implemented for obtaining lower bounds on the Chvátal–Sankoff constant. The bound $\gamma_2 \geq 0.7964$ has been obtained, which exceeds the best published result of 0.792666 (Heineman et al., 2024). An independent verification via the Monte Carlo method confirmed the correctness of the results and yielded the empirical estimate $\gamma_2 \approx 0.811$.

The method underwent six iterations of optimisation, ultimately reaching the parameters $N = 12$, $MAX\_SH = 30$, $K = 40{,}000$, requiring the processing of approximately $2 \cdot 10^{13}$ arithmetic operations.

The main directions for future research are as follows:

1. **Scaling up the parameters:** deploying computations on clusters and supercomputers will allow the use of larger values of K and MAX_SH, which, according to preliminary estimates, may yield $\gamma_2 \geq 0.80$.

2. **Algorithmic optimisations:** pruning masks for which the greedy strategy is provably optimal (when $\mathrm{LCS}_{\mathrm{visible}} \geq N/2$) can accelerate the computation by one to two orders of magnitude.

3. **Parallelisation:** the algorithm admits parallelisation over the masks *maskA* and *maskB* for a fixed L.

4. **Generalisation:** adapting the method to larger alphabets ($|\Sigma| > 2$) and investigating upper bounds.

References

[1] V. Chvátal, D. Sankoff, "Longest common subsequences of two random sequences," *Journal of Applied Probability*, vol. 12, no. 2, pp. 306–315, 1975.

[2] V. Dančík, "Expected length of longest common subsequences," Ph.D. dissertation, University of Warwick, 1994.

[3] V. Dančík, M. Paterson, "Upper bounds for the expected length of a longest common subsequence of two binary sequences," *Random Structures & Algorithms*, vol. 6, no. 4, pp. 449–458, 1995.

[4] G. S. Lueker, "Improved bounds on the average length of longest common subsequences," in *Proc. 12th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 130–136, 2003.

[5] G. S. Lueker, "Improved bounds on the average length of longest common subsequences," *Journal of the ACM*, vol. 56, no. 3, pp. 17:1–17:38, 2009.

[6] G. T. Heineman, C. Miller, D. Reichman, A. Salls, G. Sárközy, D. Soiffer, "Improved lower bounds on the expected length of longest common subsequences," *arXiv:2407.10925*, 2024.

[7] A. Tiskin, "The Chvátal–Sankoff problem as a problem of symbolic dynamics," *Journal of Mathematical Sciences*, vol. 528, pp. 214–237, 2023.

[8] J. M. Steele, "An Efron–Stein inequality for nonsymmetric statistics," *The Annals of Statistics*, vol. 14, no. 2, pp. 753–758, 1986.

[9] M. Kiwi, M. Loebl, J. Matoušek, "Expected length of the longest common subsequence for large alphabets," *Advances in Mathematics*, vol. 197, no. 2, pp. 480–498, 2005.

[10] D. S. Hirschberg, "Algorithms for the longest common subsequence problem," *Journal of the ACM*, vol. 24, no. 4, pp. 664–675, 1977.

Word Count: 1980